FC Fluxo de Caixa

DOCUMENTO 00

Visão geral e guia de execução

O que a solução faz, como rodar localmente e o mapa dos requisitos do desafio

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

Documentos entregáveis (PDF) 3
Como o requisito é atendido 3
Stack 4
Estrutura do repositório 5
Pré-requisitos 5
Subir no local 5
Testes 7
Deploy na AWS 8
Documentação 8
Saúde dos processos 9
Como ler se o tempo for curto 10
Mapa dos requisitos do desafio 11

FONTE: README.MD

Fluxo de Caixa

Sistema para um comerciante registrar créditos e débitos e consultar o saldo diário consolidado. A escrita do lançamento não depende do serviço de relatório: se o consolidado cair, o caixa continua aceitando movimento, e o saldo alcança o fato quando o consumidor volta.

O requisito que organiza a arquitetura é este: o controle de lançamentos permanece disponível se o consolidado diário cair; em pico, o consolidado atende **50 requisições por segundo** com **no máximo 5% de perda** (meta interna: 1%).

Documentos entregáveis (PDF)

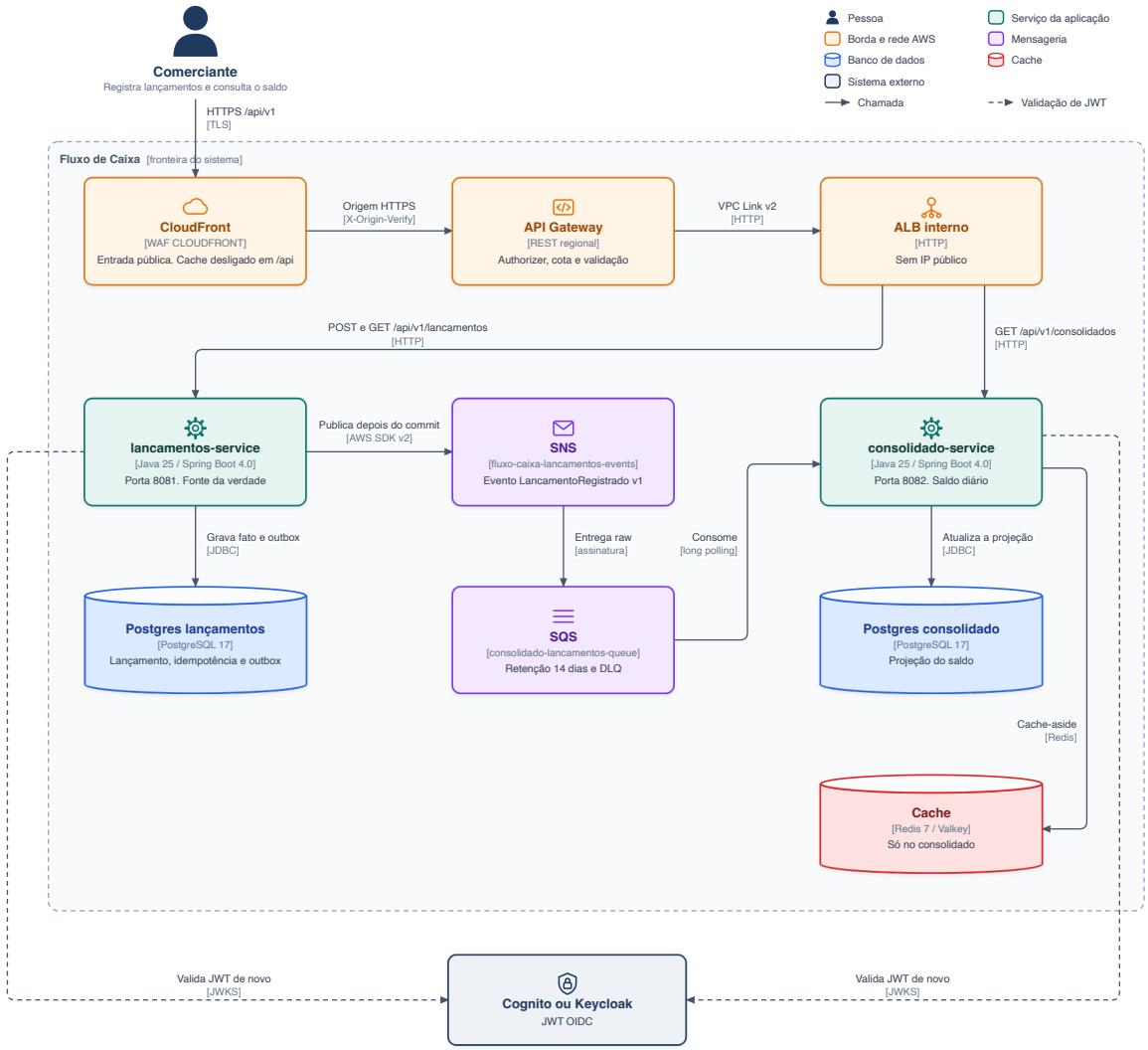
A documentação final do desafio está em PDF, diagramada e com sumário, na pasta `documentos-entregaveis-desafio/`. Para ler tudo de uma vez, abra a documentação completa. Os documentos também estão separados por tema (requisitos, DDD, arquitetura e C4, ADRs, segurança, observabilidade, custos, transição, testes, entre outros).

A pasta `documentos-brutos/` contém os **documentos brutos**: as fontes em Markdown a partir das quais os PDFs são gerados (`make pdf`). Elas ficam versionadas junto com o código e prevalecem se houver divergência.

Como o requisito é atendido

O `lancamentos-service` não chama o `consolidado-service`. O POST grava o lançamento e o evento na mesma transação (tabela `outbox_eventos`) e responde. Um relay publica no tópico cujo ARN vem de `LANCAMENTOS_TOPIC_ARN`. O consolidado consome a fila em `CONSOLIDADO_QUEUE_URL` (retenção de 14 dias, DLQ após 5 tentativas), aplica a soma uma única vez por `eventId` e serve a leitura por uma projeção em Postgres mais cache. No laboratório os nomes são `fluxo-caixa-lancamentos-events` e `consolidado-lancamentos-queue`. Na AWS o nome leva o prefixo do ambiente (`fluxo-caixa-dev-` ou `fluxo-caixa-prod-`); o serviço não tem nome de fila fixo no código. A conta de capacidade (Little: 2,8 requisições em voo no pico, duas tasks, cerca de 17% de utilização, 0,16 conexão de banco ativa) está em documentos-brutos/01-contexto-e-requisitos.md.

Fluxo de Caixa — containers



Stack

Peça	Escolha
Aplicação	Java 25, Spring Boot 4.0.9, Resilience4j 2.4.0, Maven wrapper, um projeto por serviço
Arquitetura	Dois microsserviços, hexagonal, CQRS, outbox, consumidor idempotente
Dados	PostgreSQL 17, um banco por serviço, Flyway. RDS Multi-AZ em produção. Réplica de leitura desligada até gatilho
Mensageria	SNS + SQS + DLQ. LocalStack no laboratório. AWS SDK v2. Trace propagado por atributo
Cache	Redis 7 local, Valkey no ElastiCache, só no consolidado
Segurança	OAuth2/OIDC. Cognito na AWS, Keycloak no local. JWT validado no serviço. Rotação de senha do RDS
Borda	CloudFront + WAF + VPC Origin + ALB interno, região `us-east-1`
Execução	ECS Fargate (ARM). EKS é evolução, não o deploy de hoje

Peça	Escolha
Observabilidade	OpenTelemetry. ADOT, CloudWatch e X-Ray na AWS. Grafana LGTM no local
Infra	Terraform (provider AWS 6.x), ambientes `dev`, `hml` e `prod`, state em S3

Licença de runtime: nenhuma paga. A estimativa em `us-east-1`, com os três ambientes ligados o mês inteiro e o API Gateway REST ligado, fica em cerca de **USD 143/mês** (dev), **USD 356/mês** (hml) e **USD 777/mês** (prod). A mesma produção em `sa-east-1`, com o mesmo unitário de request da REST API (não reconsultado naquela região nesta passagem), sairia cerca de **USD 1.467/mês**. Local continua **USD 0**. Detalhe em documentos-brutos/08-custos.md.

Estrutura do repositório

```
fluxo-caixa/
├── README.md
├── Makefile
├── documentos-entregaveis-desafio/ # entregáveis finais em PDF
├── documentos-brutos/              # documentos brutos (fontes Markdown) e blueprint
├── services/
│   ├── lancamentos-service/        # porta 8081
│   └── consolidado-service/        # porta 8082
├── infra/
│   ├── terraform/                  # dev, hml e prod
│   └── local/                      # docker-compose
├── tests/
│   ├── load/                       # k6
│   └── e2e/                        # smoke.sh e resiliencia.sh
├── tools/pdf/                      # gerador dos PDFs (pandoc + Chrome)
└── .github/workflows/
```

O contrato compartilhado (paths, evento, nomes de fila, variáveis, portas) está em documentos-brutos/blueprint-tecnico.md. A narrativa de arquitetura começa em documentos-brutos/README.md.

Pré-requisitos

- Java 25 (Temurin).
- Docker com Compose v2.
- Make.
- Opcional: k6 (`make load`), Terraform e AWS CLI (deploy).

O Maven não precisa estar instalado: cada serviço traz `mvnw`.

Subir no local

Na raiz do repositório:

```
make infra-up
make up
```

`make infra-up` sobe a infraestrutura local. `make up` sobe os dois serviços.

Serviço	URL
Keycloak	http://localhost:8080 (realm `fluxo-caixa`)
Lançamentos	http://localhost:8081
Consolidado	http://localhost:8082
Swagger lançamentos	http://localhost:8081/swagger-ui.html
Swagger consolidado	http://localhost:8082/swagger-ui.html
OpenAPI	http://localhost:8081/v3/api-docs e http://localhost:8082/v3/api-docs
Grafana (otel-lgtm)	http://localhost:3000
Postgres	localhost:5432, bancos `lancamentos` e `consolidado`
Redis	localhost:6379
LocalStack	http://localhost:4566

Usuário de laboratório: `comerciante1` / `comerciante1`, com `comerciante_id` `11111111-1111-1111-1111-111111111111`. Quando o Grafana pedir senha, a padrão da imagem `grafana/otel-lgtm` é `admin` / `admin`.

Para derrubar: `make down`.

Token

```
TOKEN=$(make token)
```

`make token` imprime o access token do `comerciante1` no stdout (client `fluxo-caixa-app`, direct access grant no Keycloak local).

Registrar um crédito

```
curl -sS -X POST http://localhost:8081/api/v1/lancamentos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{"tipo":"CREDITO","valor":150.00,"dataLancamento":"2026-11-10","descricao":"Venda
      balcão","categoria":"VENDAS"}'
```

A primeira resposta é **201**. Repetir com a mesma `Idempotency-Key` e o mesmo corpo devolve **200** e o mesmo recurso, sem segundo lançamento. O `comercianteId` não vai no JSON: o serviço lê o claim do token.

Consultar o lançamento e o dia

```
curl -sS http://localhost:8081/api/v1/lancamentos/COLE_O_ID \
  -H "Authorization: Bearer $TOKEN"

curl -sS "http://localhost:8081/api/v1/lancamentos?data=2026-11-10&page=0&size=50" \
  -H "Authorization: Bearer $TOKEN"
```

Estornar

```
curl -sS -X POST http://localhost:8081/api/v1/lancamentos/COLE_O_ID/estornos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"motivo":"Valor digitado errado"}'
```

O estorno é um lançamento novo, de tipo oposto, na mesma data de competência. O original não muda.

Saldo consolidado

O saldo é eventual (meta p95 abaixo de 5 s). Se o GET vier zerado logo depois do POST, espere um instante e repita.

```
curl -sS http://localhost:8082/api/v1/consolidados/diario/2026-11-10 \
  -H "Authorization: Bearer $TOKEN"

curl -sS "http://localhost:8082/api/v1/consolidados/diario?inicio=2026-11-01&fim=2026-11-10" \
  -H "Authorization: Bearer $TOKEN"
```

Dia sem movimento responde **200** com zeros. Período maior que 366 dias responde **400** `application/problem+json`.

Escopos exigidos: `lancamentos:write`, `lancamentos:read`, `consolidado:read`. No local o prefixo é vazio. Na AWS a autoridade leva o prefixo `fluxo-caixa/`.

Testes

```
make test          # unidade, ArchUnit e integração (Testcontainers)
make smoke         # jornada ponta a ponta
make resiliencia   # consolidado parado, POST segue, fila recupera
make load          # k6, opcional
make tf-validate   # Terraform fmt e validate
```

O que cada camada cobre está em documentos-brutos/11-testes.md.

Deploy na AWS

Produção em `us-east-1`: CloudFront com WAF, API Gateway REST regional com WAF próprio e VPC Link v2 até um ALB interno, VPC em três zonas, ECS Fargate com no mínimo duas tasks por serviço, RDS PostgreSQL separado e Multi-AZ para cada serviço, ElastiCache com réplica, SNS/SQS/DLQ, Cognito, Secrets Manager com rotação e ADOT. Há também `dev` e `hml`, menores. `enable_api_gateway = false` tira o gateway e volta o CloudFront à VPC Origin. O passo a passo de apply, state e variáveis fica em infra/terraform/README.md. A topologia e o DR estão em documentos-brutos/03-arquitetura/deployment-aws.md. O deploy da aplicação é rolling com circuit breaker e rollback (ADR 0020), promovendo a mesma imagem de dev para hml e prod.

Documentação

Os entregáveis em PDF estão em `documentos-entregaveis-desafio/`. A tabela abaixo aponta para os documentos brutos em Markdown, em `documentos-brutos/`, que são a fonte desses PDFs.

Documento	Conteúdo
Entregáveis em PDF	Versão final diagramada de toda a documentação
Índice	Mapa de toda a entrega
Blueprint	Contrato: nomes, portas, evento, SLOs
Requisitos	RF, RNF e capacidade
Domínio	DDD, context map, evento
Arquitetura	C4, fluxos, AWS, Structurizr
ADRs	Decisões de 0001 a 0031
RNF operacionais	SLO, error budget, k6
Segurança	STRIDE, OWASP API, LGPD, rotação
Observabilidade	Trace ponta a ponta, alarmes, runbooks
Custos	USD e BRL, dev, hml e prod
Transição	Strangler a partir do legado
Evoluções	O que ficou de fora e por quê
Testes	Pirâmide e comandos
Desempenho	Índices, réplica, cache, lake
ECS e EKS	Critério, stack e preço de observar
CI/CD	Pipeline e ferramentas

Saúde dos processos

O ALB usa readiness. No local:

- http://localhost:8081/actuator/health/liveness
- http://localhost:8081/actuator/health/readiness
- http://localhost:8082/actuator/health/readiness

Métrica Prometheus: `/actuator/prometheus` em cada serviço (não publicada na internet em produção).

FONTE: DOCUMENTOS-BRUTOS/README.MD

Documentação de entrega — Fluxo de Caixa

Esta pasta guarda os **documentos brutos** (fontes em Markdown). A versão final em PDF, diagramada, está em `documentos-entregaveis-desafio/` e é gerada a partir daqui com `make pdf`.

Índice da documentação de arquitetura do desafio de Arquiteto de Soluções. O contrato executável (nomes, portas, eventos, recursos AWS e metas de SLO) está no blueprint técnico. Esta pasta explica o porquê, os trade-offs e como operar a solução. Em caso de divergência de contrato, vale o blueprint.

Documento	O que responde
01 — Contexto e requisitos	Negócio, glossário, RF com aceite, RNF mensuráveis, cálculo de capacidade do pico de 50 req/s
02 — Domínio (DDD)	Capacidades, subdomínios, context map, event storming, agregados, modelos de dados, contrato do evento
03 — Arquitetura	C4, Structurizr, estilos, fluxos e implantação AWS
04 — ADRs	Decisões no formato MADR, de 0001 a 0031, datadas em 2026-11-10
05 — Requisitos não funcionais	SLI/SLO, error budget, escala, resiliência, plano k6, rastreabilidade
06 — Segurança	STRIDE, OAuth2, multi-tenant, OWASP API, criptografia, LGPD, rotação de segredos
07 — Observabilidade	Trace W3C ponta a ponta, logs, RED/USE, alarmes, runbooks, Grafana e X-Ray
08 — Custos	dev, hml e prod em `us-east-1`, a mesma conta em `sa-east-1`, EKS e SaaS
09 — Arquitetura de transição	Strangler Fig a partir de planilha ou ERP monolítico
10 — Evoluções futuras	Roadmap: multi-moeda, fechamento, Open Finance, multi-região, Kafka
11 — Testes	Pirâmide, como executar smoke, resiliência e carga
12 — Desempenho e dados	Índices, keyset, réplica, cache, Aurora, DynamoDB, lake
13 — ECS e EKS	Quando o EKS passa a valer, stack open source, Karpenter, preço de observar
14 — CI/CD e qualidade	Pipeline, ferramentas e o que muda com o EKS

Como ler se o tempo for curto

1. README da raiz — como subir e o que a solução faz.
2. Arquitetura-alvo — estilos e o isolamento do RNF crítico.
3. ADR 0002, ADR 0003 e ADR 0004 — o núcleo da decisão.
4. Custos — o número mensal e o que realmente o compõe (NAT, endpoints, RDS), em `us-east-1`.

Mapa dos requisitos do desafio

Exigência	Onde está atendida
Mapeamento de domínios e capacidades	02
Refinamento de requisitos funcionais e não funcionais	01, 05
Desenho da solução (arquitetura-alvo)	03
Justificativa de ferramentas, tecnologias e estilo	04, 03 — arquitetura-alvo
Testes	11, `tests/e2e`, `tests/load`
README com execução local	README da raiz
Documentação no repositório	esta pasta
Arquitetura de transição (diferencial)	09
Estimativa de custos e licenças (diferencial)	08
Monitoramento e observabilidade (diferencial)	07
Critérios de segurança para integração (diferencial)	06
RNF crítico: lançamentos independe do consolidado; 50 req/s com perda ≤ 5%	01 § análise de capacidade, 05